MarkSync | Task 2

Chaos testing strategy and AI engineering log
Advanced Software Engineering | 4 November 2026

1. System under test and correctness contract

MarkSync uses Yjs text replicas over authenticated WebSockets, with SQLite storing both raw Markdown and encoded CRDT state. Three editors and one read-only observer use the same Replica class as the browser and connect to the production HTTP/WebSocket application. Yjs resolves concurrent text operations; the application supplies delivery, access control, persistence and recovery. Named cursor/selection presence uses relative positions and is deliberately transient.

Property: once editing stops and communication becomes reliable, every authorised replica must converge on exactly the same text and operation frontier, matching the persisted server state. The reference Y.Doc receives each generated edit before network loss, so equal-but-incomplete clients cannot pass by merely copying an incomplete server result. Local edit operations are also checked against ordinary string operations. This verifies transport and preservation under Yjs semantics, not an independently reimplemented proof of Yjs.

2. Fault model, generation and recovery

fast-check generates 8-35 Unicode insert/delete/replace operations per scenario across three editors, plus two coverage probes. Character data includes ASCII, whitespace, accented text, Chinese and emoji. Edits remain concurrent because replicas can work before receiving one another's updates. One editor restores its queued state halfway through, exercising tab-refresh recovery.

A seeded virtual-time queue intercepts real application messages before send and after receipt. It injects 1-12 tick delays, approximately 23% random loss and 20% duplication, with due-time ordering allowing later operations to arrive first. Explicit probes discard the first upstream update, allow another to reach the server, and discard a downstream sync and acknowledgement. Every property case asserts that both loss directions, acknowledgement loss and delay occurred. A separate scenario verifies actual duplication and reordering.

At quiescence, faults stop and delayed messages drain. Clients retry pending batches and request missing Yjs state through ordinary resync messages; three reliable repair rounds must converge and clear pending queues. The harness never installs the oracle into a client. The browser invokes the same repair-message method every three seconds, including on an open socket. Test-only fixed replica IDs and per-receiver processing barriers make seeded schedules replayable; ping/pong barriers transfer no document state.

3. Reproduction, CI and scope

From **app/**: **pnpm install --frozen-lockfile**, then **pnpm test:chaos** (Windows: pnpm.cmd). Set **CHAOS_SEED** and optionally **CHAOS_RUNS**; use the **CHAOS_PATH** printed by fast-check to replay a shrunk failure. Default: 100 scenarios. CI runs seeds 20261104 and 11042026 with 100 scenarios each inside the **checks** job. A failure fails that job. The repository owner must require this status check before merging; workflow configuration alone is not enforcement.

This is bounded evidence, not a mathematical proof or load benchmark. Recovery assumes eventual delivery, valid permissions and configured size limits. Permanent partitions, revoked access, total browser-storage loss, server crash atomicity and multi-server scaling are outside these claims. Overlapping edits may converge to text requiring human revision. The independent reference uses Yjs too, so a shared library bug is a remaining limitation.

AI engineering & debugging log

Actual findings from AI-assisted implementation and test execution; formal group review is a separate step.

Instance 1 | A live connection could remain permanently out of sync

Generated approach: earlier preparation retried pending operations on typing or reconnect and fetched missing state on join. Its generated merge test eventually delivered the store's final state directly to clients. That could validate CRDT merging while concealing a delivery-protocol gap.

Problem and observed evidence: losing the final outgoing edit, a remote delta or an acknowledgement without closing the socket left no new event to trigger recovery. The new real-WebSocket regression initially failed with **"valid recovery messages must be accepted"** and repeated **"Invalid event or document."** responses: the server had no resync endpoint. A working CRDT did not make the application recover missing messages.

Resolution: added authenticated, rate/size-bounded state-vector resync and periodic pending-update retry, with recovery requests shared between the UI and tests. A lost first response also retries join. The test now repairs only through the real protocol. A negative control disables repair and must detect a lost edit; it cannot pass by converging all clients to an incomplete server snapshot. Two browser sessions with their first outgoing edits deliberately dropped also recovered automatically and showed named cursors.

Critical evaluation: the earlier test was useful for merging and serialisation, but insufficient evidence for distributed recovery. Claims must distinguish the library's merge rules from the application's delivery guarantees. Directly forcing a final state is not a substitute for exercising the recovery path.

Instance 2 | A loss test did not actually exercise downstream loss

Generated approach: the first new regression sent two edits through a seeded random-drop channel and asserted that both upstream and downstream messages had been lost.

Problem and observed evidence: after resync was implemented, seed 42 failed at **assert(n.metrics.droppedDown > 0)**. Both outgoing edits could disappear before the server broadcast anything, so there was no downstream message to drop. Meanwhile, a 25-scenario property run passed. A green result and a probability setting did not establish the intended fault coverage.

Resolution: added explicit coverage probes and measured loss counters, including acknowledgement loss. Retained random latency, loss, duplication and operation ordering around the probes. Added a scenario requiring reordering/duplication and a replay test comparing final text plus fault metrics across two identical runs. Broadcast barriers and test-only fixed Yjs IDs remove incidental scheduling/identity variation from the seeded experiment.

Critical evaluation: tests must demonstrate the fault they claim to simulate. The protected probe is disclosed rather than calling the channel uniformly random. Negative controls and fault counters make superficial tests easier to reject. No fabricated peer feedback or invented AI research conversation is used as evidence.

Evidence and handover

Sources: **test/chaos/network.ts**, **test/chaos/sync.chaos.ts**, **public/collaboration.js**, **src/presence.ts**; detailed log: **docs/TASK2-AI-LOG.md**. Existing tests additionally cover permissions, real-socket presence/cursors, persistence, duplicate delivery and personal undo. Read **docs/TASK2.md** for assumptions, demonstration and requirement mapping.

Submission: after peer review and passing remote CI, merge the PR, tag the final main commit, then email the tag, this report and the merged PR URL. This report does not assert that review, merge, tagging or email submission has happened.